

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a7b979af0e35bab63.jsonl`  
- SHA-256: `d9af1f1c394e8535dd3bbe50782c3c90113b17018e9d0793ed9470b09dc4debe`  
- Source modified: `2026-07-02T02:52:10+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_41c8dc8a-8a4`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T02:47:11.982Z)

CONTEXT (read carefully):

- You are one auditor in a multi-agent tech-debt/critical-fix audit of the khelsutra-guru multi-repo workspace at C:/Users/avidu/Projects/khelsutra-guru. Today is 2026-07-02. All git repos were just pulled to current origin master/main.
- Repos: badminton-highlight-indexer (the ENGINE, Python, ~1500 tests, org Khelsutra), khelsutra (SPA web/ + marketing site/, TS/React, org avidu), sports-data-collector, rally-annotator, sports-obsreport (small Python repos). Plus two NON-git dirs: "Annotation Setup", khelsutra-screencast.
- HARD RULES: READ-ONLY. Do NOT modify/create/delete any file, do NOT create branches/worktrees, do NOT pip install, do NOT run the full test suite (CI is trusted on master). You MAY run read-only git and gh commands (gh is authenticated), ruff/mypy IF already installed, and fast read-only scripts.
- EVERY claim must cite file:line (or a gh URL). A claim without file:line evidence is a hypothesis - mark it as such or drop it. The codebase drifts between sessions: re-verify anything you believe from docs against actual code.
- OWNER-GATED areas (flag as owner_gated=true, still report): alpha/serving go-live + Cloudflare dashboard work, Studio P10/P11 (corpus promotion + train/reeval), product/strategy/licensing decisions, real GPU/cloud spend, counsel ToS/DPA, repo rename off "badminton".
- Known-open items from project memory (VERIFY current state, don't trust blindly): engine PR #390 (D1 split main.py, open since 06-26) · audit doc docs/CODE_CLEANUP_AUDIT_2026-06-24.md remaining items D1, D13 (ByteTrack->trackers), Phase-3 (B3/B4/B6/B7/B8/D5/D6/D8-D10/03/04) · docs/DECISION_SNAPSHOT_REGRESSION.md next=P1 · docs/GOLDEN_REGRESSION_FIXTURES.md backlog M4/P0-rename/01/02 · engine issues #340 (Gemini re-bill), #332 (NSFW preflight), #349 (CPU-bound decode), #384 (CI single-source) · khelsutra issues #97-#114 UX/alpha backlog · DO droplet RETIRED 2026-06-25 (docs may still reference it; api.khelsutra.guru CNAME/CF-Access cleanup was pending).
- Categorize each finding: critical-fix (latent bug/correctness/data-loss/cost-leak), security, tech-debt (structure/duplication/dead code), test-gap, ci-infra, doc-rot, hygiene.
- Severity: P0 = latent bug or cost/data/security risk reachable in normal use; P1 = high-value debt blocking velocity or a stale-open thread; P2 = worthwhile cleanup; P3 = nice-to-have (report max 3 of these).
- Be selective: max ~12 findings, each genuinely worth acting on. Note tracked_in when an issue/PR/doc already tracks it (tracked is NOT done - still report). Your final message is data for an orchestrator, not prose for a human.

YOUR DIMENSION: full audit of sports-obsreport. Repo: C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport.

Small private Python lib (observability/reporting; HEAD = v0.1.3 configurable item_noun). It is a git+ssh dependency of the engine (C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer) - the engine suite skips 12 tests when it's absent. Audit: how the engine pins it (grep pyproject/requirements in the engine for obsreport - pinned tag/commit or floating branch? floating = P1 reproducibility debt), release/tag hygiene (git tag -l vs versions), CI state, test coverage, API surface stability, README accuracy.

2. user (2026-07-02T02:47:18.708Z)

```
1f72c1c feat: configurable highlights item_noun (v0.1.3)
2f5a214 feat: spec-level customer_verdict override (v0.1.2)
cba67f2 feat: customer_message ? curated customer-facing flag text (v0.1.1)
7c40654 fix: correct setuptools build-backend (underscore) so editable/pip install works
d3e2a51 C4: RunRecorder + stable-diff + JSON Schema contract
911adf7 C3: spec-driven markdown renderers (technical + customer) + safe IO
918a728 C2: report config language ? spec loader + restricted rule engine
687b52e C1: scaffold obsreport ? typed envelope + deterministic JSON renderer
v0.1.0
v0.1.1
v0.1.2
v0.1.3
* main
  remotes/origin/main
```

3. user (2026-07-02T02:47:20.099Z)

```
total 29
drwxr-xr-x 1 avidu 197609  0 Jun  7 13:13 .
drwxr-xr-x 1 avidu 197609  0 Jun 29 20:32 ..
drwxr-xr-x 1 avidu 197609  0 Jul  2 08:17 .git
-rw-r--r-- 1 avidu 197609 169 Jun  7 12:37 .gitignore
drwxr-xr-x 1 avidu 197609  0 Jun  7 12:39 .pytest_cache
-rw-r--r-- 1 avidu 197609 1066 Jun  7 12:37 LICENSE
-rw-r--r-- 1 avidu 197609 1589 Jun  7 12:38 README.md
-rw-r--r-- 1 avidu 197609  764 Jun  7 13:17 pyproject.toml
drwxr-xr-x 1 avidu 197609  0 Jun  7 12:49 schema
drwxr-xr-x 1 avidu 197609  0 Jun  7 12:49 scripts
drwxr-xr-x 1 avidu 197609  0 Jun  7 12:47 specs
drwxr-xr-x 1 avidu 197609  0 Jun  7 12:37 src
drwxr-xr-x 1 avidu 197609  0 Jun  7 13:17 tests
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/.gitignore
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/.pytest_cache/.gitignore
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/.pytest_cache/CACHEDIR.TAG
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/.pytest_cache/README.md
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/.pytest_cache/v/cache/nodeids
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/LICENSE
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/pyproject.toml
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/README.md
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/schema/report_envelope.schema.json
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/scripts/gen_schema.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/specs/example.yaml
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/diff.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/envelope.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/io.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/recorder.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/render/json_render.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/render/markdown.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/render/__init__.py
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/render/__pycache__/json_render.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/render/__pycache__/markdown.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/render/__pycache__/__init__.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/rules.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/spec.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/version.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/src/obsreport/__init__.py
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/__pycache__/diff.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/__pycache__/envelope.cpython-313.pyc
```

```
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/__pycache__/io.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/__pycache__/recorder.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/__pycache__/rules.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/__pycache__/spec.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/__pycache__/version.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/src/obsreport/__pycache__/__init__.cpython-313.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_customer_message.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_diff.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_envelope.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_io.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_recorder.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_render_json.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_render_markdown.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_rules.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_schema.py
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport/tests/test_spec.py
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/tests/__pycache__/test_customer_message.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/tests/__pycache__/test_diff.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/tests/__pycache__/test_envelope.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/tests/__pycache__/test_io.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/tests/__pycache__/test_recorder.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/tests/__pycache__/test_render_json.cpython-313-pytest-9.0.3.pyc
/c/Users/avidu/Projects/khelsutra-guru/sports-
obsreport/tests/__pycache__/test_render_markdown.cpython-313-pytest-9.0.3.pyc
```

4. user (2026-07-02T02:47:30.190Z)

```
v0.1.3
1f72c1c2dd471d4e3ed3130f0a4040a6d4813042
821a664f5d2861397d5de7ebe803e72f71e9964a
.
..
.git
.gitignore
.pytest_cache
LICENSE
README.md
pyproject.toml
schema
scripts
specs
src
tests
```

5. user (2026-07-02T02:47:30.203Z)

```
1      [build-system]
2      requires = ["setuptools>=61.0.0"]
3      build-backend = "setuptools.build_meta"
4
5      [project]
6      name = "obsreport"
7      version = "0.1.3"
```

```

8     description = "Reusable per-run observability report toolkit: a small config-language
for emitting technical + customer-friendly reports across data/video pipelines."
9     readme = "README.md"
10    requires-python = ">=3.9"
11    license = { text = "MIT" }
12    authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
13    dependencies = [
14        "pydantic>=2.0.0",
15        "jinja2>=3.1.0",
16        "pyyaml>=6.0",
17    ]
18
19    [project.optional-dependencies]
20    dev = [
21        "pytest>=7.0.0",
22        "pytest-cov>=4.0.0",
23    ]
24
25    [tool.setuptools.packages.find]
26    where = ["src"]
27
28    [tool.pytest.ini_options]
29    pythonpath = ["src"]
30    testpaths = ["tests"]
31

```

6. user (2026-07-02T02:47:30.204Z)

```

1     # obsreport
2
3     A small, reusable toolkit for emitting rich per-run observability reports** from
4     data/video pipelines ? designed so multiple tools produce reports in the same
5     shape**, making them diffable across runs and across tools.
6
7     It is deliberately not a heavy data-quality framework (Great Expectations,
8     Evidently, Soda, ?). Those model tabular datasets; this models a single pipeline
9     run as an ordered list of heterogeneous stages* plus a declarative
10    "footgun ? flag" rule layer. It is built only on permissive primitives
11    (`pydantic`, `Jinja2`, `PyYAML`) so it is safe for commercial use.
12
13    ## What you get
14
15    - One typed envelope (`ReportEnvelope`) that is the single source of truth.
16      Both the technical and the customer-friendly report render from the same object,
17      so they can never disagree on facts.
18    - A declarative report config language (YAML): describe report sections and
19      rules* that turn known failure modes into flags, each citing a FAQ section so a
20      reader can self-debug. *(spec.py / rules.py ? added in C2.)*
21    - Renderers**: deterministic JSON (machine source of truth, today) + Markdown
22      technical & customer views *(C3)*.
23    - A RunRecorder** to capture stages in-flight, plus post-hoc harvest helpers,
24      with writes that never raise so reporting can't break the host run *(C4)*.
25
26    ## Status
27
28    Early scaffolding. Phase C1 (envelope + JSON renderer) is in place; C2?C4 follow.
29    See the consuming repos (`badminton-highlight-indexer`, `sports-data-collector`)
30    for the integration.
31
32    ## Develop
33
34    ```bash
35    pip install -e ".[dev]"
36    pytest
37    ```

```

7. user (2026-07-02T02:47:34.608Z)

```

Found 33 files
tests\test_api_endpoints.py
docs\CODE_MAP.md
backend\main.py
README.md
.github\workflows\ci.yml
tests\test_reporting_harvest.py
tests\test_reporting_spec_faq.py
tests\test_reporting.py
tests\test_run_signals.py
tests\test_person_detector.py
tests\test_async_jobs.py
deploy\digitalocean\bootstrap.sh
docs\CODE_CLEANUP_AUDIT_2026-06-24.md
pyproject.toml
requirements.txt
run_all_tests.py
backend\reporting\run_signals.py
backend\reporting\__init__.py
backend\reporting\harvest.py
docs\archives\decisions\DECISIONS.md
docs\ROADMAP.md
docs\PER_VIDEO_WORKSPACE.md
docs\PACKAGING_PLAN.md
docs\COMMERCIALIZATION.md
docs\BACKLOG.md
docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md
docs\archives\DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md
docs\archives\HARDENING_LOOP_HANDOFF-COMPLETED-2026-06-14.md
backend\reporting\spec.yaml
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md
docs\archives\quality-iterations\2026-06-initial\README.md
docs\archives\checkpoints\2026-06-logging-reporting\README.md

```

8. user (2026-07-02T02:47:45.373Z)

```

1f72c1c2dd471d4e3ed3130f0a4040a6d4813042
7a340d578f97ace3e1bd66bb9e63dacccecb23f      refs/tags/v0.1.0
7c40654cf3cf233bb1daf7512066b8ce3cf1a5a8      refs/tags/v0.1.0^{
2f98bc5fa9b34f632f8135ea20c00f596b3996ef      refs/tags/v0.1.1
cba67f29d261903e03026fe5fef5a56ade63860a4      refs/tags/v0.1.1^{
75aea1b807a79cf3669534808c3d48fb01753975      refs/tags/v0.1.2
2f5a214bf5051c386875a0e81bc724383b51d7e9      refs/tags/v0.1.2^{
821a664f5d2861397d5de7ebe803e72f71e9964a      refs/tags/v0.1.3
1f72c1c2dd471d4e3ed3130f0a4040a6d4813042      refs/tags/v0.1.3^{
origin https://github.com/Khelsutra/sports-obsreport.git (fetch)
origin https://github.com/Khelsutra/sports-obsreport.git (push)
1f72c1c feat: configurable highlights item_noun (v0.1.3)
2f5a214 feat: spec-level customer_verdict override (v0.1.2)
cba67f2 feat: customer_message ? curated customer-facing flag text (v0.1.1)

```

9. user (2026-07-02T02:47:45.499Z)

```

62-# Per-video observability reports (SOFT dependency: the pipeline degrades to a
63-# no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
64-# fetched over SSH; if you lack access, simply omit this group ? reporting
65-# tests skip via pytest.importorskip. For local dev against a sibling checkout:
66:# pip install -e ../sports-obsreport --no-build-isolation
67-reporting = [

```

```

68:     "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
69-]
70-
71-# `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
72-# CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
73-#
74-# `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
75-#
76-# indexer = "backend.main:main"
77-#
78-# [tool.hatch.metadata]
79-# The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
80-# reference*); hatchling requires opting in to allow those in metadata.
81-#
82-# allow-direct-references = true
83-#
84-# [tool.hatch.build.targets.wheel]

```

10. user (2026-07-02T02:47:45.954Z)

```

31-
32-# Per-video observability reports (SOFT dependency: the pipeline degrades to a
33-# no-op if absent). Shared, permissively-licensed report toolkit. Pin a tag once
34-# the private repo is published, e.g.:
35-#   obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3
36-# For local development install it editable from a sibling checkout:
37-#   pip install -e ../sports-obsreport --no-build-isolation

```

11. user (2026-07-02T02:47:47.055Z)

```

131-     # NOTE: roboflow/trackers uses bare version tags (2.5.0), NOT v-prefixed.
132-     run: python -m pip install "trackers @
133-     git+https://github.com/roboflow/trackers.git@2.5.0"
134-     - name: Install package (editable) + dev tooling
135-     # Editable PEP 621 install: runtime deps + the `dev` group
136-     # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
137-     run: python -m pip install -e .[dev]
138-     # obsreport (private, soft dep) is absent here: reporting tests skip via
139-     # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
140-     # is therefore calibrated against THIS lane's number (local runs with
141-     # obsreport installed measure a little higher).
142-     # Floor calibrated 2026-06-11 against this lane's measured 72% (local
143-     # with obsreport: 73%). Ratchet it UP as coverage grows; never down
144-     # without an owner decision.
145-     - name: Run full suite (with coverage floor)
146-     run: python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-
147-     fail-under=70
148-
149-# Cross-OS determinism guard for the committed golden regression fixtures
150-# (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib

```

12. user (2026-07-02T02:48:16.326Z)

```

{"defaultBranchRef":{"name":"main"},"isPrivate":true,"name":"sports-obsreport","owner":{"id":"0_
kgDOEYa5Fw","login":"Khelsutra"},"pushedAt":"2026-06-07T08:06:31Z","visibility":"PRIVATE"}
---
Khelsutra/sports-obsreport (redirect target)
---
0
---
---

```

13. user (2026-07-02T02:48:16.353Z)

```

1     """Version + report-schema version.
2

```

```
3     `SCHEMA_VERSION` is the contract version of the report ENVELOPE (bump the major
4     when a change is not backward compatible). `__version__` is the package release.
5     Consumers pin a git tag, so a schema bump can never surprise them silently.
6     """
7
8     SCHEMA_VERSION = "1.0"
9     __version__ = "0.1.3"
10
```

14. user (2026-07-02T02:48:16.374Z)

```
1     """obsreport ? reusable per-run observability reports.
2
3     Build a `ReportEnvelope` (typed, single source of truth), optionally run a
4     declarative rule spec to derive flags, then render it to JSON (machine) and
5     Markdown (technical + customer). Designed to be shared across pipelines so their
6     reports are diffable with each other and across runs.
7     """
8     from .version import SCHEMA_VERSION, __version__
9     from .envelope import (
10         Flag,
11         HighlightSummary,
12         Identity,
13         Metric,
14         ReportEnvelope,
15         Stage,
16         VideoMeta,
17         verdict_from_flags,
18     )
19     from .render.json_render import to_json
20     from .render.markdown import render_customer, render_technical
21     from .spec import Clause, ReportSpec, Rule, Section, load_spec, load_spec_dict,
22     load_spec_text
23     from .rules import apply_rules, run_spec
24     from .io import WriteResult, report_paths, write_report
25     from .recorder import RunRecorder
26     from .diff import stable_dict, stable_json
27
28     __all__ = [
29         "SCHEMA_VERSION",
30         "__version__",
31         "ReportEnvelope",
32         "Identity",
33         "VideoMeta",
34         "Stage",
35         "Metric",
36         "Flag",
37         "HighlightSummary",
38         "verdict_from_flags",
39         "to_json",
40         "ReportSpec",
41         "Section",
42         "Rule",
43         "Clause",
44         "load_spec",
45         "load_spec_text",
46         "load_spec_dict",
47         "apply_rules",
48         "run_spec",
49         "render_technical",
50         "render_customer",
51         "write_report",
52         "report_paths",
53         "WriteResult",
54         "RunRecorder",
55     ]
```

```

54         "stable_json",
55         "stable_dict",
56     ]
57

```

15. user (2026-07-02T02:48:29.137Z)

```

1     """The rule engine: evaluate a spec's rules against an envelope to derive flags.
2
3     Everything here is pure and dependency-free. A missing path makes a clause False
4     for every operator except ``is_missing`` ? so a typo or an absent stage can never
5     *accidentally* fire a footgun flag.
6     """
7     from __future__ import annotations
8
9     from typing import Any, Dict, List
10
11     from .envelope import Flag, ReportEnvelope
12     from .spec import Clause, Rule, ReportSpec
13
14
15     class _Missing:
16         """Sentinel for an unresolved path (distinct from a real ``None`` value)."""
17
18         _singleton = None
19
20         def __repr__(self) -> str: # pragma: no cover - cosmetic
21             return "<MISSING>"
22
23
24     MISSING = _Missing()
25
26
27     def to_rule_context(env: ReportEnvelope) -> Dict[str, Any]:
28         """Project an envelope into an ergonomic dict for dotted-path lookups.
29
30         Lists that you'd want to address by name are re-keyed: ``stages`` by stage
31         name, and each stage's ``metrics`` by metric key. So a rule can say
32         ``stages.segmentation.metrics.segment_count`` or
33         ``stages.ai_handoff.details.api_key_present``.
34         """
35         stages: Dict[str, Any] = {}
36         for s in env.stages:
37             stages[s.name] = {
38                 "status": s.status,
39                 "duration_s": s.duration_s,
40                 "messages": list(s.messages),
41                 "details": dict(s.details),
42                 "metrics": {m.key: m.value for m in s.metrics},
43             }
44         return {
45             "identity": env.identity.model_dump(mode="json"),
46             "video_meta": env.video_meta.model_dump(mode="json"),
47             "highlights": env.highlights.model_dump(mode="json") if env.highlights else {},
48             "verdict": env.verdict,
49             "extras": dict(env.extras),
50             "stages": stages,
51             "flags": [f.model_dump(mode="json") for f in env.flags],
52         }
53
54
55     def get_path(data: Any, dotted: str) -> Any:
56         """Navigate ``data`` by a dotted path; return MISSING if any hop is absent.
57         Numeric path parts index into lists."""
58         cur = data

```



```

59     for part in dotted.split("."):
60         if isinstance(cur, dict):
61             if part in cur:
62                 cur = cur[part]
63             else:
64                 return MISSING
65         elif isinstance(cur, list):
66             try:
67                 idx = int(part)
68             except ValueError:
69                 return MISSING
70             if -len(cur) <= idx < len(cur):
71                 cur = cur[idx]
72             else:
73                 return MISSING
74         else:
75             return MISSING
76     return cur
77
78
79 def _ordered(op: str, a: Any, b: Any) -> bool:
80     try:
81         if op == "gt":
82             return a > b
83         if op == "ge":
84             return a >= b
85         if op == "lt":
86             return a < b
87         if op == "le":
88             return a <= b
89     except TypeError:
90         return False
91     return False
92
93
94 def eval_clause(ctx: Dict[str, Any], clause: Clause) -> bool:
95     actual = get_path(ctx, clause.path)
96     op = clause.op
97
98     if op == "is_missing":
99         return actual is MISSING
100    if op == "is_present":
101        return actual is not MISSING
102    if actual is MISSING:
103        # Every other operator is False against a missing path (no accidental fires).
104        return False
105
106    if op == "eq":
107        return actual == clause.value
108    if op == "ne":
109        return actual != clause.value
110    if op in ("gt", "ge", "lt", "le"):
111        return _ordered(op, actual, clause.value)
112    if op == "in":
113        try:
114            return actual in clause.value
115        except TypeError:
116            return False
117    if op == "contains":
118        try:
119            return clause.value in actual
120        except TypeError:
121            return False
122    if op in ("eq_path", "ne_path"):
123        other = get_path(ctx, clause.value) if isinstance(clause.value, str) else

```

```

MISSING
124         if other is MISSING:
125             return False
126         return actual == other if op == "eq_path" else actual != other
127     return False
128
129
130 def eval_rule(ctx: Dict[str, Any], rule: Rule) -> bool:
131     """A rule fires when ALL its clauses hold (logical AND)."""
132     return all(eval_clause(ctx, c) for c in rule.when)
133
134
135 def _evidence(ctx: Dict[str, Any], rule: Rule) -> Dict[str, Any]:
136     """Snapshot the values that triggered the rule, so the flag is self-explaining
137     in the report (MISSING rendered as None for JSON-safety)."""
138     ev: Dict[str, Any] = {}
139     for c in rule.when:
140         val = get_path(ctx, c.path)
141         ev[c.path] = None if val is MISSING else val
142     return ev
143
144
145 def apply_rules(env: ReportEnvelope, spec: ReportSpec) -> List[Flag]:
146     """Return the flags a spec's rules produce for an envelope (no mutation)."""
147     ctx = to_rule_context(env)
148     flags: List[Flag] = []
149     for rule in spec.rules:
150         if eval_rule(ctx, rule):
151             flags.append(
152                 Flag(
153                     code=rule.code,
154                     severity=rule.severity,
155                     message=rule.message,
156                     stage=rule.stage,
157                     faq_ref=rule.faq_ref,
158                     customer_message=rule.customer_message,
159                     evidence=_evidence(ctx, rule),
160                 )
161             )
162     return flags
163
164
165 def run_spec(env: ReportEnvelope, spec: ReportSpec) -> ReportEnvelope:
166     """Apply a spec's rules to ``env`` in place, then recompute the verdict."""
167     env.flags.extend(apply_rules(env, spec))
168     env.recompute_verdict()
169     return env
170

```

16. user (2026-07-02T02:48:29.614Z)

```

1     """File output: the path convention + safe writes.
2
3     Convention (next to the video by default):
4         /videos/match.mp4 -> /videos/match.report.json    (machine, source of truth)
5                             /videos/match.report.md       (technical, debuggable)
6                             /videos/match.summary.md      (customer-friendly)
7
8     Overridable with ``output_dir``. For remote/no-local-file runs (e.g. the
9     collector before download), output falls back to ``output_dir`` or
10    ``reports/<video_id>/``.
11
12    THE CARDINAL RULE: writing a report must NEVER break the host pipeline run.
13    Every write is wrapped; failures are collected into a ``WriteResult`` and
14    returned, never raised.

```

```

15     """
16     from __future__ import annotations
17
18     import os
19     from dataclasses import dataclass, field
20     from typing import Dict, List, Optional
21
22     from .envelope import ReportEnvelope
23     from .render.json_render import to_json
24     from .render.markdown import render_customer, render_technical
25     from .spec import ReportSpec
26
27
28     @dataclass
29     class WriteResult:
30         ok: bool
31         paths: Dict[str, str] = field(default_factory=dict)
32         errors: List[str] = field(default_factory=list)
33
34
35     def _base(video_path: Optional[str], output_dir: Optional[str], video_id: Optional[str],
stem: Optional[str]):
36         """Return (directory, basename) for the report files."""
37         if stem:
38             name = stem
39         elif video_path:
40             name = os.path.splitext(os.path.basename(video_path))[0]
41         elif video_id:
42             name = video_id
43         else:
44             name = "report"
45
46         if output_dir:
47             return output_dir, name
48         if video_path:
49             return os.path.dirname(os.path.abspath(video_path)) or ".", name
50         # remote / no local file
51         return os.path.join("reports", video_id or "run"), name
52
53
54     def report_paths(
55         video_path: Optional[str] = None,
56         output_dir: Optional[str] = None,
57         video_id: Optional[str] = None,
58         stem: Optional[str] = None,
59     ) -> Dict[str, str]:
60         """Resolve the three report file paths (no IO)."""
61         directory, name = _base(video_path, output_dir, video_id, stem)
62         return {
63             "json": os.path.join(directory, f"{name}.report.json"),
64             "technical_md": os.path.join(directory, f"{name}.report.md"),
65             "customer_md": os.path.join(directory, f"{name}.summary.md"),
66         }
67
68
69     def _write_text(path: str, text: str) -> None:
70         """Atomic write: temp file in the same dir, then os.replace. Patchable in tests."""
71         directory = os.path.dirname(path) or "."
72         os.makedirs(directory, exist_ok=True)
73         tmp = f"{path}.tmp"
74         with open(tmp, "w", encoding="utf-8", newline="\n") as f:
75             f.write(text)
76         os.replace(tmp, path)
77
78

```

```

79     def write_report(
80         envelope: ReportEnvelope,
81         spec: Optional[ReportSpec] = None,
82         *,
83         video_path: Optional[str] = None,
84         output_dir: Optional[str] = None,
85         video_id: Optional[str] = None,
86         stem: Optional[str] = None,
87         write_customer: bool = True,
88     ) -> WriteResult:
89         """Render + write json / technical md / (optional) customer md. Never raises."""
90         result = WriteResult(ok=True)
91         try:
92             paths = report_paths(
93                 video_path=video_path,
94                 output_dir=output_dir,
95                 video_id=video_id or envelope.identity.video_id,
96                 stem=stem,
97             )
98         except Exception as e: # pragma: no cover - defensive
99             return WriteResult(ok=False, errors=[f"path resolution failed: {e}"])
100
101         jobs = [
102             ("json", paths["json"], lambda: to_json(envelope)),
103             ("technical_md", paths["technical_md"], lambda: render_technical(envelope,
104 spec))]
105         if write_customer:
106             jobs.append(("customer_md", paths["customer_md"], lambda:
107 render_customer(envelope, spec)))
108
109         for key, path, render in jobs:
110             try:
111                 _write_text(path, render())
112                 result.paths[key] = path
113             except Exception as e:
114                 result.ok = False
115                 result.errors.append(f"{key}: {e}")
116         return result

```

17. user (2026-07-02T02:48:37.451Z)

```

1     """Typed report envelope ? the single source of truth for one observability report.
2
3     A report describes ONE processed video / one pipeline run as an ordered list of
4     heterogeneous *stages*, plus video metadata, derived *flags* (warnings/errors),
5     a customer-safe *highlight summary*, and an overall *verdict*.
6
7     Both the technical and the customer-friendly renderers consume this SAME object,
8     so the two reports can never disagree on facts. This module is pure data + a few
9     pure helpers ? there is NO file/network IO here (see `io.py`) and nothing runs at
10    import time.
11    """
12    from __future__ import annotations
13
14    from typing import Any, Dict, List, Literal, Optional
15
16    from pydantic import BaseModel, Field
17
18    from .version import SCHEMA_VERSION
19
20    # --- closed vocabularies (kept as Literals so typos fail fast in consumers) ---
21    Audience = Literal["technical", "customer", "both"]
22    StageStatus = Literal["ok", "warn", "error", "skipped", "not_run"]
23    Severity = Literal["error", "warn", "info"]
24    Verdict = Literal["pass", "pass_with_warnings", "fail"]

```

```

25     FpsSource = Literal["probed", "fallback", "declared", "unknown"]
26     VideoIdKind = Literal["file_md5", "human_id", "unknown"]
27
28
29     class Metric(BaseModel):
30         """A single named measurement attached to a stage.
31
32         `audience` controls which report it surfaces in: technical-only by default;
33         use ``"both"`` for things a customer should also see (e.g. duration).
34         """
35
36         key: str
37         value: Any = None
38         unit: Optional[str] = None
39         audience: Audience = "technical"
40
41
42     class Stage(BaseModel):
43         """One step the video went through (validate, segment, stitch, acquire, ...)."""
44
45         name: str
46         status: StageStatus = "ok"
47         started_at: Optional[str] = None
48         duration_s: Optional[float] = None
49         metrics: List[Metric] = Field(default_factory=list)
50         details: Dict[str, Any] = Field(default_factory=dict)
51         messages: List[str] = Field(default_factory=list)
52
53         def add_metric(
54             self,
55             key: str,
56             value: Any,
57             unit: Optional[str] = None,
58             audience: Audience = "technical",
59         ) -> "Stage":
60             """Append a metric and return self (chainable)."""
61             self.metrics.append(Metric(key=key, value=value, unit=unit, audience=audience))
62             return self
63
64
65     class Flag(BaseModel):
66         """A derived warning/error ? the debugging intelligence of the report.
67
68         `faq_ref` points at the README/FAQ section a reader should open to self-debug
69         (e.g. ``"README#Q7"``). It is REQUIRED for error/warn severities; this is
70         enforced by `ReportEnvelope.lint`, not by the constructor, so harvest code can
71         build a flag incrementally before deciding its citation.
72         """
73
74         code: str
75         severity: Severity
76         message: str
77         stage: Optional[str] = None
78         faq_ref: Optional[str] = None
79         evidence: Dict[str, Any] = Field(default_factory=dict)
80         # Plain-language phrasing for the customer report. The customer view shows ONLY
81         # flags that set this (so internal/technical findings never leak); the technical
82         # view always uses `message`.
83         customer_message: Optional[str] = None
84
85
86     class Identity(BaseModel):
87         """Who/what this report is about. `video_id_kind` records the id SEMANTICS so a
88         cross-tool diff (indexer file-MD5 vs collector human id) never mis-joins."""
89

```

```

90     tool: str
91     schema_version: str = SCHEMA_VERSION
92     tool_version: Optional[str] = None
93     video_id: Optional[str] = None
94     video_id_kind: VideoIdKind = "unknown"
95     video_path: Optional[str] = None
96     video_url: Optional[str] = None
97     generated_at: Optional[str] = None
98     run_id: Optional[str] = None
99
100
101 class VideoMeta(BaseModel):
102     """Probed/declared media facts. `fps_source` distinguishes a real probe from a
103     silent fallback (a known footgun the report exists to surface)."""
104
105     duration_s: Optional[float] = None
106     fps: Optional[float] = None
107     fps_source: FpsSource = "unknown"
108     resolution: Optional[str] = None
109     width: Optional[int] = None
110     height: Optional[int] = None
111     container: Optional[str] = None
112     size_mb: Optional[float] = None
113
114
115 class HighlightSummary(BaseModel):
116     """Customer-safe, meta-level summary of what the tool produced (no internals)."""
117
118     segment_count: int = 0
119     total_highlight_s: float = 0.0
120     coverage_pct: Optional[float] = None
121     notes: List[str] = Field(default_factory=list)
122
123
124 def verdict_from_flags(flags: List[Flag]) -> Verdict:
125     """Pure verdict derivation: any error -> fail; any warn -> pass_with_warnings."""
126     severities = {f.severity for f in flags}
127     if "error" in severities:
128         return "fail"
129     if "warn" in severities:
130         return "pass_with_warnings"
131     return "pass"
132
133
134 class ReportEnvelope(BaseModel):
135     """The whole report. Serialize with `obsreport.to_json` (machine source of
136     truth) or render via the markdown renderers (human/customer views)."""
137
138     identity: Identity
139     video_meta: VideoMeta = Field(default_factory=VideoMeta)
140     stages: List[Stage] = Field(default_factory=list)
141     flags: List[Flag] = Field(default_factory=list)
142     highlights: Optional[HighlightSummary] = None
143     verdict: Verdict = "pass"
144     extras: Dict[str, Any] = Field(default_factory=dict)
145
146     # --- pure helpers (no IO) ---
147     def get_stage(self, name: str) -> Optional[Stage]:
148         for s in self.stages:
149             if s.name == name:
150                 return s
151         return None
152
153     def recompute_verdict(self) -> "ReportEnvelope":
154         """Set `verdict` from current flags. Call in finalize, after rules run."""

```

```

155         self.verdict = verdict_from_flags(self.flags)
156         return self
157
158     def lint(self) -> List[str]:
159         """Structural self-checks. Empty list == clean. Used by tests and the
160         recorder's finalize to guarantee a well-formed, debuggable report."""
161         issues: List[str] = []
162         for f in self.flags:
163             if f.severity in ("error", "warn") and not f.faq_ref:
164                 issues.append(
165                     f"flag '{f.code}' has severity '{f.severity}' but no faq_ref "
166                     f"(error/warn flags must cite a README/FAQ section)"
167                 )
168         expected = verdict_from_flags(self.flags)
169         if self.verdict != expected:
170             issues.append(
171                 f"verdict '{self.verdict}' does not match flag-derived '{expected}' "
172                 f"(did you forget recompute_verdict())?"
173             )
174         return issues
175

```

18. user (2026-07-02T02:48:38.074Z)

```

1     """RunRecorder ? capture an envelope incrementally during a pipeline run.
2
3     A tool creates a recorder at the top of a run, opens a ``stage(...)`` per pipeline
4     step (auto-timed; exceptions are captured into the stage, NOT re-raised, so
5     reporting can't crash the run), fills in video metadata / highlights / manual
6     flags, then ``finalize()`` (runs the spec's rules + computes the verdict) and
7     ``write()`` (emits the files). The clock is injectable so tests are deterministic.
8
9     Tool-specific *post-hoc harvest* (rebuilding an envelope from a DB / artifacts)
10    lives in each consuming repo ? it just drives this same recorder API.
11    """
12    from __future__ import annotations
13
14    import time
15    from contextlib import contextmanager
16    from datetime import datetime, timezone
17    from typing import Any, Callable, Dict, Iterator, Optional
18
19    from .envelope import Flag, HighlightSummary, Identity, ReportEnvelope, Stage
20    from .io import WriteResult, write_report
21    from .rules import run_spec
22    from .spec import ReportSpec
23
24
25    def _utc_now_iso() -> str:
26        return datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")
27
28
29    class RunRecorder:
30        def __init__(
31            self,
32            tool: str,
33            *,
34            tool_version: Optional[str] = None,
35            video_id: Optional[str] = None,
36            video_id_kind: str = "unknown",
37            video_path: Optional[str] = None,
38            video_url: Optional[str] = None,
39            run_id: Optional[str] = None,
40            spec: Optional[ReportSpec] = None,
41            now: Callable[[], str] = _utc_now_iso,

```

```

42         monotonic: Callable[[], float] = time.perf_counter,
43     ) -> None:
44         self._now = now
45         self._monotonic = monotonic
46         self.spec = spec
47         self._finalized = False
48         self.envelope = ReportEnvelope(
49             identity=Identity(
50                 tool=tool,
51                 tool_version=tool_version,
52                 video_id=video_id,
53                 video_id_kind=video_id_kind, # type: ignore[arg-type]
54                 video_path=video_path,
55                 video_url=video_url,
56                 run_id=run_id,
57                 generated_at=now(),
58             )
59         )
60
61     # --- media / highlights ---
62     def set_video_meta(self, **kw: Any) -> "RunRecorder":
63         self.envelope.video_meta = self.envelope.video_meta.model_copy(update=kw)
64         return self
65
66     def set_highlights(self, **kw: Any) -> "RunRecorder":
67         cur = self.envelope.highlights or HighlightSummary()
68         self.envelope.highlights = cur.model_copy(update=kw)
69         return self
70
71     # --- stages ---
72     def get_or_create_stage(self, name: str) -> Stage:
73         st = self.envelope.get_stage(name)
74         if st is None:
75             st = Stage(name=name)
76             self.envelope.stages.append(st)
77         return st
78
79     @contextmanager
80     def stage(self, name: str, *, capture_exceptions: bool = True) -> Iterator[Stage]:
81         """Time a pipeline step. On exception, mark the stage 'error' and record the
82         message; re-raise only if ``capture_exceptions=False``."""
83         st = self.get_or_create_stage(name)
84         st.started_at = self._now()
85         t0 = self._monotonic()
86         try:
87             yield st
88         except Exception as e: # noqa: BLE001 - we deliberately swallow to protect the
89             st.status = "error"
90             st.messages.append(f"{type(e).__name__}: {e}")
91             if not capture_exceptions:
92                 raise
93         finally:
94             st.duration_s = round(self._monotonic() - t0, 6)
95
96     # --- flags ---
97     def add_flag(
98         self,
99         code: str,
100         severity: str,
101         message: str,
102         *,
103         faq_ref: Optional[str] = None,
104         stage: Optional[str] = None,
105         evidence: Optional[Dict[str, Any]] = None,

```



```

106         customer_message: Optional[str] = None,
107     ) -> "RunRecorder":
108         self.envelope.flags.append(
109             Flag(
110                 code=code,
111                 severity=severity, # type: ignore[arg-type]
112                 message=message,
113                 faq_ref=faq_ref,
114                 stage=stage,
115                 evidence=evidence or {},
116                 customer_message=customer_message,
117             )
118         )
119         return self
120
121     # --- finalize / write ---
122     def finalize(self) -> ReportEnvelope:
123         """Apply the spec's rules (if any) and compute the verdict. Idempotent."""
124         if not self._finalized:
125             if self.spec is not None:
126                 run_spec(self.envelope, self.spec)
127             else:
128                 self.envelope.recompute_verdict()
129             self._finalized = True
130         return self.envelope
131
132     def write(self, **io_kwargs: Any) -> WriteResult:
133         """Finalize then write all report files. Never raises (see io.write_report)."""
134         self.finalize()
135         kwargs: Dict[str, Any] = dict(
136             video_path=self.envelope.identity.video_path,
137             video_id=self.envelope.identity.video_id,
138         )
139         kwargs.update(io_kwargs)
140         return write_report(self.envelope, self.spec, **kwargs)
141

```

19. user (2026-07-02T02:48:47.933Z)

```

1     """The report config language: a YAML spec describing report *sections* (layout)
2     and *rules* (footgun -> flag).
3
4     A spec is pure data validated into pydantic models. Specs are authored by us (one
5     per tool, under ``specs/``), so we validate strictly and fail fast: every
6     actionable rule (error/warn) MUST cite a FAQ section, and every rule MUST have at
7     least one condition. That guarantee is what makes the resulting reports
8     self-debuggable.
9     """
10    from __future__ import annotations
11
12    from pathlib import Path
13    from typing import Any, Dict, List, Literal, Optional, Union
14
15    import yaml
16    from pydantic import BaseModel, Field, model_validator
17
18    from .envelope import Audience, Severity
19
20    # Comparison operators the rule engine understands. Deliberately a closed,
21    # sub-Turing set ? there is NO expression grammar and NO eval anywhere.
22    ClauseOp = Literal[
23        "eq", "ne", "gt", "ge", "lt", "le",
24        "in", "contains",
25        "is_missing", "is_present",
26        "eq_path", "ne_path",

```

```

27     ]
28
29
30     class Clause(BaseModel):
31         """One condition: compare the value at ``path`` against ``value``.
32
33         ``path`` is a dotted accessor into the rule context (see ``rules.to_rule_context``),
34         e.g. ``stages.ai_handoff.metrics.segment_count``. For ``eq_path``/``ne_path``,
35         ``value`` is itself a dotted path (compare two fields)."""
36
37         path: str
38         op: ClauseOp
39         value: Any = None
40
41
42     class Rule(BaseModel):
43         """A footgun detector. If every clause in ``when`` holds, emit a flag."""
44
45         code: str
46         severity: Severity
47         message: str
48         when: List[Clause] = Field(default_factory=list)
49         faq_ref: Optional[str] = None
50         stage: Optional[str] = None
51         customer_message: Optional[str] = None # if set, this finding also shows (gently)
to customers
52
53         @model_validator(mode="after")
54         def _validate(self) -> "Rule":
55             if self.severity in ("error", "warn") and not self.faq_ref:
56                 raise ValueError(
57                     f"rule '{self.code}' has severity '{self.severity}' but no faq_ref "
58                     f"('{actionable flags must cite a README/FAQ section so a reader can self-
debug)'"
59                 )
60             if not self.when:
61                 raise ValueError(
62                     f"rule '{self.code}' has no conditions; a rule must have >=1 clause "
63                     f"('{use is_present/is_missing for existence checks})"
64                 )
65             return self
66
67
68     SectionKind = Literal[
69         "identity", "video_meta", "highlights", "verdict", "flags", "stages", "stage"
70     ]
71
72
73     class Section(BaseModel):
74         """A block in the rendered report. The renderer walks these in order."""
75
76         id: str
77         title: str
78         audience: Audience = "technical"
79         kind: SectionKind = "stages"
80         stage: Optional[str] = None # required when kind == "stage"
81         metrics: Optional[List[str]] = None # optional whitelist of metric keys to show
82         item_noun: str = "highlight" # noun for a HighlightSummary item (e.g. "rally",
"point")
83
84         @model_validator(mode="after")
85         def _validate(self) -> "Section":
86             if self.kind == "stage" and not self.stage:
87                 raise ValueError(f"section '{self.id}' is kind=stage but names no 'stage'")
88             return self

```

```

89
90
91     class ReportSpec(BaseModel):
92         name: str
93         title: Optional[str] = None
94         sections: List[Section] = Field(default_factory=list)
95         rules: List[Rule] = Field(default_factory=list)
96         # Optional per-tool phrasing of the customer-facing verdict, keyed by verdict
97         # (pass / pass_with_warnings / fail). Lets each tool speak to its own audience
98         # (e.g. a highlight viewer vs an annotation vendor). Falls back to a generic
99         default.
100         customer_verdict: Optional[Dict[str, str]] = None
101
102     def load_spec_dict(data: dict) -> ReportSpec:
103         return ReportSpec.model_validate(data or {})
104
105
106     def load_spec_text(text: str) -> ReportSpec:
107         return load_spec_dict(yaml.safe_load(text) or {})
108
109
110     def load_spec(path: Union[str, Path]) -> ReportSpec:
111         return load_spec_text(Path(path).read_text(encoding="utf-8"))
112

```

20. user (2026-07-02T02:48:48.827Z)

```

1     """Regenerate the checked-in JSON Schema for the report envelope.
2
3     Run after changing envelope.py:  python scripts/gen_schema.py
4     The schema is the published contract; test_schema.py fails if it drifts.
5     """
6     import json
7     import sys
8     from pathlib import Path
9
10    ROOT = Path(__file__).resolve().parent.parent
11    sys.path.insert(0, str(ROOT / "src"))
12
13    from obsreport.envelope import ReportEnvelope # noqa: E402
14
15    OUT = ROOT / "schema" / "report_envelope.schema.json"
16
17
18    def schema_text() -> str:
19        return json.dumps(ReportEnvelope.model_json_schema(), indent=2, sort_keys=True) +
20    "\n"
21
22    def main() -> None:
23        OUT.parent.mkdir(parents=True, exist_ok=True)
24        OUT.write_text(schema_text(), encoding="utf-8")
25        print(f"wrote {OUT}")
26
27
28    if __name__ == "__main__":
29        main()
30

```

21. user (2026-07-02T02:48:49.581Z)

Found 13 files

C:\Users\avidu\Projects\khehsutra-guru\sports-data-collector\tests\test_reporting_noraise.py
C:\Users\avidu\Projects\khehsutra-guru\sports-data-collector\src\reporting__init__.py

```
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\src\reporting\adapter.py
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\pyproject.toml
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\sports-data-
collector\tests\test_reporting_faq.py
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\sports-data-
collector\tests\test_reporting.py
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\sports-data-
collector\src\reporting\adapter.py
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\sports-data-
collector\src\reporting\spec.yaml
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\sports-data-
collector\src\reporting\__init__.py
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\sports-data-
collector\pyproject.toml
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\tests\test_reporting_faq.py
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\tests\test_reporting.py
C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector\src\reporting\spec.yaml
```

22. user (2026-07-02T02:49:01.281Z)

```
31-]
32-# Per-video observability reports (SOFT dependency: ingestion degrades to a no-op
33-# if absent). Shared, permissively-licensed report toolkit. Pin a tag once the
34-# private repo is published, e.g.:
35-#  obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3
36-# For local development install it editable from a sibling checkout:
37-#  pip install -e ../sports-obsreport --no-build-isolation
38-report = [
39:  "obsreport>=0.1.3",
40-]
```

23. user (2026-07-02T02:49:04.587Z)

```
schema in sync: True
version: 0.1.3
```

24. user (2026-07-02T02:49:04.607Z)

```
1      """Stable serialization for cross-run / cross-tool diffing.
2
3      The plain JSON is the machine source of truth, but it carries volatile fields
4      (timestamps, run id, per-stage durations) that change every run and would swamp a
5      diff. ``stable_json`` strips those so two runs of the same logical video diff only
6      where the *substance* differs ? which is the whole point of "evaluate output
7      differences."
8      """
9      from __future__ import annotations
10
11      import json
12      from typing import Any, Iterable, Set
13
14      from .envelope import ReportEnvelope
15
16      VOLATILE_KEYS: Set[str] = {"generated_at", "run_id", "started_at", "duration_s"}
17
18
19      def _strip(obj: Any, keys: Set[str]) -> Any:
20          if isinstance(obj, dict):
21              return {k: _strip(v, keys) for k, v in obj.items() if k not in keys}
22          if isinstance(obj, list):
23              return [_strip(v, keys) for v in obj]
24          return obj
25
26
```

```

27     def stable_dict(envelope: ReportEnvelope, extra_volatile: Iterable[str] = ()) -> dict:
28         keys = VOLATILE_KEYS | set(extra_volatile)
29         return _strip(envelope.model_dump(mode="json"), keys)
30
31
32     def stable_json(envelope: ReportEnvelope, indent: int = 2, extra_volatile: Iterable[str]
33 = ()) -> str:
34         return json.dumps(stable_dict(envelope, extra_volatile), indent=indent,
35 sort_keys=True, ensure_ascii=False)
36

```

25. user (2026-07-02T02:49:17.315Z)

```

404
{"message": "Not Found"}

```

26. user (2026-07-02T02:49:17.339Z)

```

1     """Markdown renderers ? technical and customer views of the SAME envelope.
2
3     Section selection/order/titles come from the report spec (or sensible defaults).
4     The technical view shows everything (stages, all flags with code+evidence+faq_ref);
5     the customer view shows only video metadata, meta-level highlights, and the
6     human-readable text of actionable flags ? never internal codes, evidence, or raw
7     stage dumps. Rendering is pure and deterministic (good for snapshot diffs).
8     """
9     from __future__ import annotations
10
11     import os
12     from typing import List, Optional
13
14     from ..envelope import Flag, ReportEnvelope, Stage
15     from ..spec import ReportSpec, Section
16
17     # Default layout when a spec ships no sections. Covers both audiences; the
18     # renderer filters by the requested audience.
19     DEFAULT_SECTIONS: List[Section] = [
20         Section(id="run", title="Run", kind="identity", audience="both"),
21         Section(id="video", title="Video", kind="video_meta", audience="both"),
22         Section(id="highlights", title="Highlights", kind="highlights",
23 audience="customer"),
24         Section(id="stages", title="Processing stages", kind="stages",
25 audience="technical"),
26         Section(id="findings", title="Findings", kind="flags", audience="technical"),
27         Section(id="issues", title="Things to know", kind="flags", audience="customer"),
28         Section(id="verdict", title="Verdict", kind="verdict", audience="both"),
29     ]
30
31     _VERDICT_LABEL = {
32         "pass": "PASS",
33         "pass_with_warnings": "PASS (with warnings)",
34         "fail": "FAIL",
35     }
36
37     # A customer should never see a bare "FAIL" ? for them the verdict is about whether
38     # their highlights are usable, framed plainly, with no internal blame.
39     _VERDICT_CUSTOMER = {
40         "pass": "? All good ? your highlights are ready.",
41         "pass_with_warnings": "? Your highlights are ready. (A couple of minor notes
42 above.)",
43         "fail": "?? We ran into a problem processing this video, so some highlights may be "
44             "missing. See the notes above, or contact support.",
45     }
46
47     _SEVERITY_MARK = {"error": "[ERROR]", "warn": "[WARN]", "info": "[info]"}
48

```

```

45 # --- small formatters -----
46 def _num(v) -> str:
47     if isinstance(v, float):
48         return f"{v:.2f}".rstrip("0").rstrip(".")
49     return str(v)
50
51
52 def _secs(v) -> str:
53     try:
54         f = float(v)
55     except (TypeError, ValueError):
56         return "?"
57     m, s = divmod(int(round(f)), 60)
58     return f"{m}{s:02d}s" if m else f"{f:.1f}s"
59
60
61 def _kv_table(rows: List[tuple]) -> str:
62     rows = [(k, v) for k, v in rows if v is not None and v != ""]
63     if not rows:
64         return "_ (none)_"
65     out = ["| Field | Value |", "| --- | --- |"]
66     out += [f"| {k} | {v} |" for k, v in rows]
67     return "\n".join(out)
68
69
70 # --- per-kind builders -----
71 def _identity(env: ReportEnvelope, audience: str) -> str:
72     i = env.identity
73     if audience == "customer":
74         name = os.path.basename(i.video_path) if i.video_path else (i.video_url or
75 i.video_id or "?")
76         return _kv_table([("Video", name), ("Report generated", i.generated_at)])
77     return _kv_table([
78         ("Tool", f"{i.tool} {i.tool_version or ''}.strip()),
79         ("Video id", f"{i.video_id} ({i.video_id_kind})" if i.video_id else None),
80         ("Video path", i.video_path),
81         ("Video url", i.video_url),
82         ("Run id", i.run_id),
83         ("Generated", i.generated_at),
84         ("Schema", i.schema_version),
85     ])
86
87 def _video_meta(env: ReportEnvelope, audience: str) -> str:
88     m = env.video_meta
89     if audience == "customer":
90         return _kv_table([
91             ("Duration", _secs(m.duration_s) if m.duration_s is not None else None),
92             ("Resolution", m.resolution or (f"{m.width}x{m.height}" if m.width and
93 m.height else None)),
94             ("Frame rate", f"{_num(m.fps)} fps" if m.fps else None),
95         ])
96     fps = f"{_num(m.fps)} fps ({m.fps_source})" if m.fps is not None else f"?
97 ({m.fps_source})"
98     return _kv_table([
99         ("Duration", _secs(m.duration_s) if m.duration_s is not None else None),
100         ("Resolution", m.resolution or (f"{m.width}x{m.height}" if m.width and m.height
101 else None)),
102         ("FPS", fps),
103         ("Container", m.container),
104         ("Size", f"{_num(m.size_mb)} MB" if m.size_mb is not None else None),
105     ])
106
107 def _plural(noun: str, n: int) -> str:

```

```

106         if n == 1:
107             return noun
108         return noun[:-1] + "ies" if noun.endswith("y") else noun + "s"
109
110
111     def _highlights(env: ReportEnvelope, audience: str, section: Optional[Section] = None)
-> str:
112         h = env.highlights
113         noun = section.item_noun if section else "highlight"
114         if not h:
115             return f"_(no {_plural(noun, 0)} produced)_"
116         lines = [
117             f"- **{h.segment_count}** {_plural(noun, h.segment_count)}",
118             f"- **{_secs(h.total_highlight_s)}** of play",
119         ]
120         if h.coverage_pct is not None:
121             lines.append(f"- **{_num(h.coverage_pct)}%** of the video is {_plural(noun,
2)}")
122         lines += [f"- {n}" for n in h.notes]
123         return "\n".join(lines)
124
125
126     def _stage_block(s: Stage, audience: str) -> str:
127         head = f"***{s.name}** ? `{s.status}`"
128         if s.duration_s is not None:
129             head += f" ({_secs(s.duration_s)})"
130         lines = [head]
131         metrics = [m for m in s.metrics if audience != "customer" or m.audience in
("customer", "both")]
132         for m in metrics:
133             unit = f" {m.unit}" if m.unit else ""
134             lines.append(f" - {m.key}: {_num(m.value)}{unit}")
135         if audience != "customer":
136             for k, v in s.details.items():
137                 lines.append(f" - _{k}_: {v}")
138         for msg in s.messages:
139             lines.append(f" - {msg}")
140         return "\n".join(lines)
141
142
143     def _stages(env: ReportEnvelope, audience: str, only: Optional[str] = None) -> str:
144         stages = env.stages if only is None else [s for s in env.stages if s.name == only]
145         if not stages:
146             return "_(no stages recorded)_"
147         return "\n\n".join(_stage_block(s, audience) for s in stages)
148
149
150     def _flags(env: ReportEnvelope, audience: str) -> str:
151         if audience == "customer":
152             # Only show findings explicitly curated for customers (those with a
153             # customer_message). Internal/technical flags never leak into this view.
154             actionable = [f for f in env.flags if f.severity in ("error", "warn") and
f.customer_message]
155             if not actionable:
156                 return "_Nothing needs your attention._"
157             mark = {"error": "?", "warn": "??"}
158             return "\n".join(f"- {mark[f.severity]} {f.customer_message}" for f in
actionable)
159         if not env.flags:
160             return "_No findings._"
161         blocks = []
162         for f in env.flags:
163             line = f"- {_SEVERITY_MARK[f.severity]} **{f.code}** ? {f.message}"
164             if f.faq_ref:
165                 line += f" \n ? see **{f.faq_ref}**"

```

```

166         if f.evidence:
167             ev = ", ".join(f"{k}={v}" for k, v in f.evidence.items())
168             line += f" \n ? evidence: {ev}"
169         blocks.append(line)
170     return "\n".join(blocks)
171
172
173 def _verdict(env: ReportEnvelope, audience: str, spec: Optional[ReportSpec] = None) ->
str:
174     if audience == "customer":
175         if spec and spec.customer_verdict and env.verdict in spec.customer_verdict:
176             return spec.customer_verdict[env.verdict]
177         return _VERDICT_CUSTOMER.get(env.verdict, env.verdict)
178     return f"***{_VERDICT_LABEL.get(env.verdict, env.verdict)}***"
179
180
181 _BUILDERS = {
182     "identity": _identity,
183     "video_meta": _video_meta,
184     "highlights": _highlights,
185     "flags": _flags,
186     "verdict": _verdict,
187 }
188
189
190 def _build_section(env: ReportEnvelope, section: Section, audience: str,
191                   spec: Optional[ReportSpec] = None) -> str:
192     if section.kind == "stages":
193         return _stages(env, audience)
194     if section.kind == "stage":
195         return _stages(env, audience, only=section.stage)
196     if section.kind == "verdict":
197         return _verdict(env, audience, spec)
198     if section.kind == "highlights":
199         return _highlights(env, audience, section)
200     return _BUILDERS[section.kind](env, audience)
201
202
203 def _render(env: ReportEnvelope, spec: Optional[ReportSpec], audience: str) -> str:
204     sections = spec.sections if (spec and spec.sections) else DEFAULT_SECTIONS
205     title = (spec.title if spec and spec.title else None) or f"{env.identity.tool} ?
video report"
206     if audience == "customer":
207         title += " (summary)"
208     parts = [f"# {title}", ""]
209     for sec in sections:
210         if sec.audience not in (audience, "both"):
211             continue
212         body = _build_section(env, sec, audience, spec)
213         parts.append(f"## {sec.title}")
214         parts.append("")
215         parts.append(body)
216         parts.append("")
217     return "\n".join(parts).rstrip() + "\n"
218
219
220 def render_technical(env: ReportEnvelope, spec: Optional[ReportSpec] = None) -> str:
221     return _render(env, spec, "technical")
222
223
224 def render_customer(env: ReportEnvelope, spec: Optional[ReportSpec] = None) -> str:
225     return _render(env, spec, "customer")
226

```


C:\Users\avidu\Projects\khehsutra-guru\sports-obsreport\README.md:11:(`pydantic`, `Jinja2`,
`PyYAML`) so it is safe for commercial use.
C:\Users\avidu\Projects\khehsutra-guru\sports-obsreport\pyproject.toml:15: "jinja2>=3.1.0",

28. user (2026-07-02T02:49:48.225Z)

```
..... [ 97%]  
.. [100%]  
74 passed in 0.43s
```

29. user (2026-07-02T02:49:48.663Z)

```
ruff 0.15.16  
4 |  
5 | from obsreport import Flag, Identity, ReportEnvelope, Stage, load_spec, run_spec  
  |                               ^^^^^  
6 | from obsreport.rules import MISSING, apply_rules, eval_clause, get_path, to_rule_context  
7 | from obsreport.spec import Clause  
  |  
help: Remove unused import: `obsreport.Flag`
```

Found 3 errors.

[*] 2 fixable with the `--fix` option (1 hidden fix can be enabled with the `--unsafe-fixes` option).

===

```
tests/test_customer_message.py:2  
tests/test_diff.py:2  
tests/test_envelope.py:8  
tests/test_io.py:7  
tests/test_recorder.py:6  
tests/test_render_json.py:3  
tests/test_render_markdown.py:7  
tests/test_rules.py:9  
tests/test_schema.py:1  
tests/test_spec.py:5
```

30. user (2026-07-02T02:49:57.863Z)

Exit code 1

```
src\obsreport\render\markdown.py:14:24: F401 [*] `..envelope.Flag` imported but unused  
tests\test_recorder.py:38:31: F841 Local variable `s` is assigned to but never used  
tests\test_rules.py:5:23: F401 [*] `obsreport.Flag` imported but unused
```

Found 3 errors.

[*] 2 fixable with the `--fix` option (1 hidden fix can be enabled with the `--unsafe-fixes` option).

31. user (2026-07-02T02:49:59.878Z)

```
tests/test_reporting.py:80:# (see test_reporting_harvest.py which uses pytest.importorskip).  
Those tests
```

```
tests/test_reporting_harvest.py:11:pytest.importorskip("obsreport")  
tests/test_reporting_spec_faq.py:14:pytest.importorskip("obsreport")  
4:and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The  
8:Imported only after `backend.reporting` confirms obsreport is installed, so it may  
9:import obsreport freely.  
19: import obsreport # soft dep; only needed for RunRecorder / load_spec at runtime  
21: obsreport = None # type: ignore[assignment]  
27:def load_spec() -> "obsreport.ReportSpec":  
28: return obsreport.load_spec(_SPEC_PATH)  
83:def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:  
98: rec: "obsreport.RunRecorder",  
146: rec: "obsreport.RunRecorder",  
182: rec: "obsreport.RunRecorder",  
217: spec: Optional["obsreport.ReportSpec"] = None,
```

```
220:) -> "obsreport.RunRecorder":
222:    rec = obsreport.RunRecorder(
```

32. assistant (2026-07-02T02:50:46.341Z)

Let me check the collector's CI usage of the `report` extra to gauge reachability, and verify a couple of README claims.

33. user (2026-07-02T02:50:48.157Z)

```
ci.yml
```

34. user (2026-07-02T02:50:49.552Z)

```
2026-06-07 13:17:39 +0530 1f72c1c feat: configurable highlights item_noun (v0.1.3)
2026-06-07 13:17:39 +0530 1f72c1c feat: configurable highlights item_noun (v0.1.3)
2026-06-07 13:14:16 +0530 2f5a214 feat: spec-level customer_verdict override (v0.1.2)
2026-06-07 13:03:41 +0530 cba67f2 feat: customer_message ? curated customer-facing flag text (v0.1.1)
2026-06-07 12:51:20 +0530 7c40654 fix: correct setuptools build-backend (underscore) so
editable/pip install works
2026-06-07 12:50:24 +0530 d3e2a51 C4: RunRecorder + stable-diff + JSON Schema contract
2026-06-07 12:47:42 +0530 911adf7 C3: spec-driven markdown renderers (technical + customer) +
safe IO
2026-06-07 12:43:23 +0530 918a728 C2: report config language ? spec loader + restricted rule
engine
2026-06-07 12:39:29 +0530 687b52e C1: scaffold obsreport ? typed envelope + deterministic JSON
renderer
```

35. user (2026-07-02T02:50:58.189Z)

```
18:    python -m pip install --upgrade pip
19:    pip install -e ".[dev]"
```

36. user (2026-07-02T02:52:10.089Z)

Structured output provided successfully